Создание виртуальной системы Client-Server
для передачи сообщений по протоколу МЭК 60870-5 104

Цель: отследить пакеты с данными передаваемыми по протолку МЭК 60870-5 104

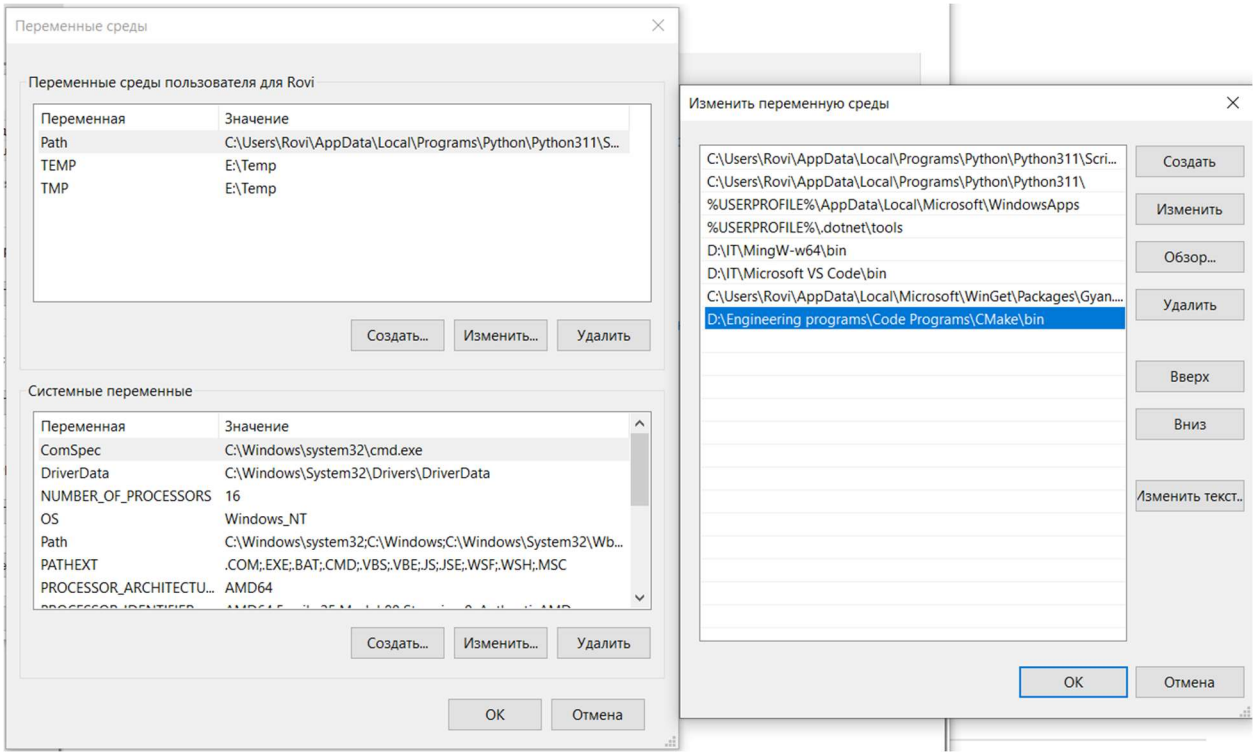
Используемое ПО:

- lib61850C - библиотека с реализованными ПО Клиент, Сервер (МЭК-104) https://github.com/mz-automation/lib60870/blob/master/lib60870-C
- WireShark - контроль сети TCP/IP
- CMake – сборка программы проекта
- Visual Studio Code (далее VS)

Ход работы:

1. Установка CMake и компиляция библиотеки lib61850C

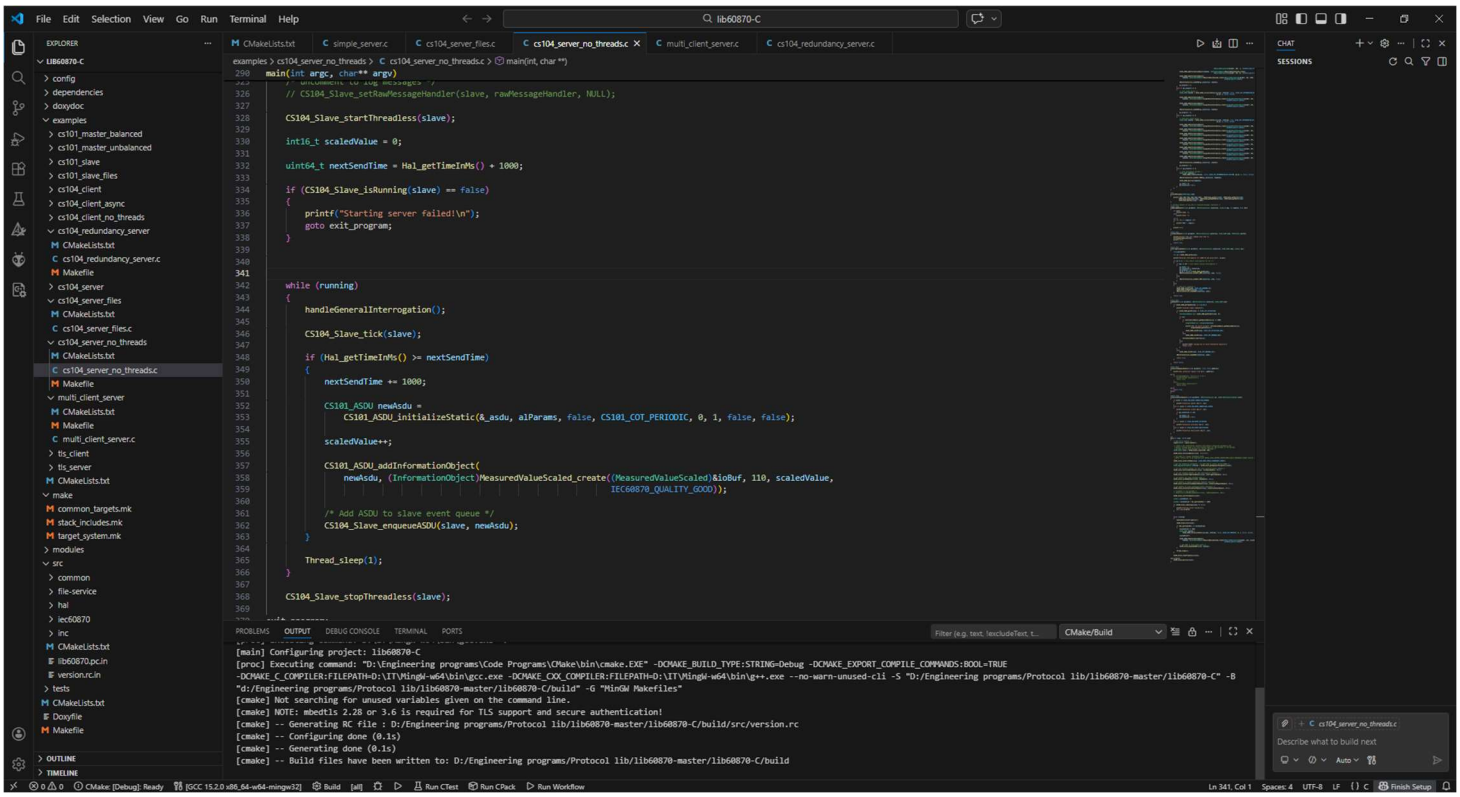
Добавляем CMake в переменную среду PATH операционной системы.



Проверяем что CMake работает, запрашивая версию ПО через консоль VS

```
PS D:\Engineering programs\Protocol lib\lib60870-master\lib60870-C> cmake --version
cmake version 4.3.0-rc1
```

Компилируем библиотеку lib61850C в VS



2. Запускаем создание проекта на основе библиотеки, с целью получения программ Server и Client работающих на протоколе IEC 60870-5-104

Так как работа VS компилятора настроена на язык C в момент создания проекта возникает ошибка которая связана с неверной инициализацией переменных в коде программы (пересечение инициализаций)

Было:

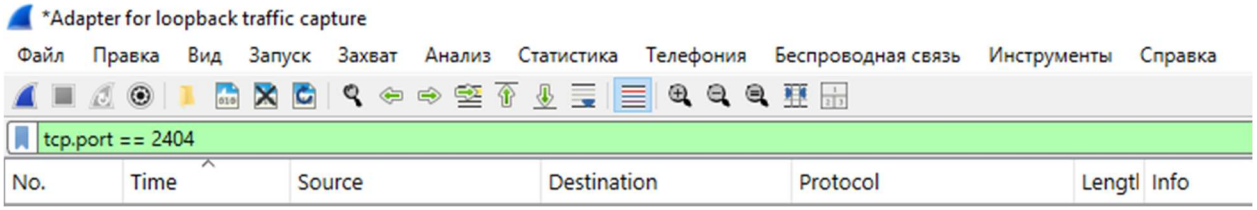
```
if (CS104_Slave_isRunning(slave) == false)
{
    printf("Starting server failed!\n");
    goto exit_program;
}
int16_t scaledValue = 0;
uint64_t nextSendTime = Hal_getTimeInMs() + 1000;
```

Стало:

```
330     int16_t scaledValue = 0;
331
332     uint64_t nextSendTime = Hal_getTimeInMs() + 1000;
333
334     if (CS104_Slave_isRunning(slave) == false)
335     {
336         printf("Starting server failed!\n");
337         goto exit_program;
338     }
```

После исправления данной проблемы создаём проект и в результате получаем файлы
simple_client.exe - Программа запуска Сервера
cs104_server.exe - Программа запуска Клиента

3. Запускаем WireSahrk и устанавливаем фильтр на отслеживание пакетов в сети, проходящие через порт 2404



4. Запускаем Сервер (эмуляция подстанции)

Открывается консоль, в которой отражены сетевые настройки передачи сервера:

t0	10 с	Таймер установки соединения (макс время на установку TCP-соединения)
t1	15 с	Таймер подтверждения (макс время между отправкой и подтверждением)
t2	10 с	Таймер отправки тестовых кадров (если нет данных, шлём тестовый кадр)
t3	20 с	Таймер подтверждения для тестовых кадров
k	12	Максимальное количество неподтверждённых I-кадров (окно отправки)
w	8	Максимальное количество принятых I-кадров до принудительного подтверждения

5. Запускаем Клиента (эмуляция диспетчерского пункта)

D:\Engineering programs\Protocol lib\lib60870-master\lib608

```
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: C_IC_NA_1(100) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 3
RECVD ASDU type: M_SP_NA_1(1) elements: 2
  single point information:
    IOA: 104 value: 1
    IOA: 105 value: 0
RECVD ASDU type: M_SP_NA_1(1) elements: 8
  single point information:
    IOA: 300 value: 1
    IOA: 301 value: 0
    IOA: 302 value: 1
    IOA: 303 value: 0
    IOA: 304 value: 1
    IOA: 305 value: 0
    IOA: 306 value: 1
    IOA: 307 value: 0
RECVD ASDU type: M_BO_NA_1(7) elements: 1
RECVD ASDU type: C_IC_NA_1(100) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
RECVD ASDU type: M_ME_NB_1(11) elements: 1
```

В консоли отображаются запросы отправляемые клиентом на сервер и ответы, получаемые от сервера.

6. Переходим в WireShark и просматриваем пакеты, которые были переданы по сети через порт 2404 по протоколу TCP и IEC 60870-5 104

tcp.port == 2404

No.	Time	Source	Destination	Protocol	Length	Info
36070	693.251999900	127.0.0.1	127.0.0.1	TCP	56	9806 → 2404 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
36071	693.252042600	127.0.0.1	127.0.0.1	TCP	56	2404 → 9806 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
36072	693.252056800	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
36073	693.260658900	127.0.0.1	127.0.0.1	IEC 60870-5-104	50	<- U (STARTDT act)
36074	693.260672000	127.0.0.1	127.0.0.1	TCP	44	2404 → 9806 [ACK] Seq=1 Ack=7 Win=2619648 Len=0
36075	693.262354100	127.0.0.1	127.0.0.1	IEC 60870-5-104	50	-> U (STARTDT con)
36076	693.262362800	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=7 Ack=7 Win=2619648 Len=0
36077	693.262377200	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	62	-> I (0,0) ASDU=1 M_ME_NB_1 Per/Cyc IOA=110
36078	693.262380300	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=7 Ack=25 Win=2619648 Len=0
36079	693.262394000	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	62	-> I (1,0) ASDU=1 M_ME_NB_1 Per/Cyc IOA=110
36080	693.262396700	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=7 Ack=43 Win=2619648 Len=0
36081	693.262409100	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	62	-> I (2,0) ASDU=1 M_ME_NB_1 Per/Cyc IOA=110
36082	693.262412100	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=7 Ack=61 Win=2619648 Len=0
36109	695.037126100	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	62	-> I (3,0) ASDU=1 M_ME_NB_1 Per/Cyc IOA=110
36110	695.037141900	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=7 Ack=79 Win=2619648 Len=0
36111	695.275736500	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	60	<- I (0,4) ASDU=1 C_IC_NA_1 Act IOA=0
36112	695.275764700	127.0.0.1	127.0.0.1	TCP	44	2404 → 9806 [ACK] Seq=79 Ack=23 Win=2619648 Len=0
36113	695.277577700	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	60	-> I (4,1) ASDU=1 C_IC_NA_1 ActCon IOA=0
36114	695.277588300	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=95 Win=2619648 Len=0
36115	695.291726700	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	74	-> I (5,1) ASDU=1 M_ME_NB_1 Inrogen IOA[3]=100,...
36116	695.291742200	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=125 Win=2619648 Len=0
36119	695.307244900	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	64	-> I (6,1) ASDU=1 M_SP_NA_1 Inrogen IOA[2]=104,...
36120	695.307251600	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=145 Win=2619648 Len=0
36123	695.323236500	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	67	-> I (7,1) ASDU=1 M_SP_NA_1 Inrogen IOA[8]=300-307
36124	695.323256600	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=168 Win=2619392 Len=0
36125	695.339262700	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	64	-> I (8,1) ASDU=1 M_BO_NA_1 Inrogen IOA=500
36126	695.339282500	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=188 Win=2619392 Len=0
36127	695.386280100	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	60	-> I (9,1) ASDU=1 C_IC_NA_1 ActTerm IOA=0
36128	695.386300800	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=204 Win=2619392 Len=0
36194	697.043190200	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	62	-> I (10,1) ASDU=1 M_ME_NB_1 Per/Cyc IOA=110
36195	697.043208300	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=222 Win=2619392 Len=0
36255	699.047504300	127.0.0.1	127.0.0.1	IEC 60870-5 ASDU	62	-> I (11,1) ASDU=1 M_ME_NB_1 Per/Cyc IOA=110
36256	699.047525000	127.0.0.1	127.0.0.1	TCP	44	9806 → 2404 [ACK] Seq=23 Ack=240 Win=2619392 Len=0

Пакеты данных под номером 30670 – 36072 – это TCP-рукопожатие (3-way handshake), которое устанавливает связь между двумя абонентами сети.

№36073 – это пакет содержащий запрос Клиента к Серверу на основе протокола МЭК-104 с командой:
U STARTDT act

где `U` — U-формат (служебная команда управления)
`STARTDT` — START Data Transfer – начало передачи циклических данных
`act` — activation (активация)

№36075 – это пакет содержащий ответ Сервера на запрос Клиента из предыдущего пакета:
U STARTDT con

где `con` — confirmation (подтверждение)

№36077 - №36308 – это пакеты данных в которых Сервер отправляет данные Клиенту о всех измерениях и состояниях дискретных выходов

Формат кадра (APCI)

1. **I-формат** — информационные кадры (с данными)
2. **S-формат** — подтверждения (сколько кадров принято)
3. **U-формат** — управление (STARTDT, STOPDT, TESTFR)

Передача данных по протоколу включает в себя следующие блоки

<- от Клиента к Серверу

-> от Сервера к Клиенту

Направление передачи	Формат кадра	Адрес ASDU	Тип ASDU (данные)	Причина передачи	Адрес датчика	Значение (данные)	Примечание
<-			U (STARTDT)	act			Клиент-Сервер: начало передачи данных
->			U (STARTDT)	con			Клиент-Сервер: подтверждение начала передачи данных
…							
->	I (2,0)	1	M_ME_NB_1 (11)	Per/Cyc (1)	I0A = 110	2	Передача циклических данных
->	I (3,0)	1	M_ME_NB_1 (11)	Per/Cyc (1)	I0A = 110	3	<...>
<-	I (0,4)	1	C_IC_NA_1	Act	IOA = 0		Отправка команды на выдачу всех дампов данных сервера + подтверждение полученных кадров 4 шт.
->	I (4,1)	1	C_IC_NA_1	ActCon	I0A = 0	3	подтверждение полученной команды
->	I (5,1)	1	M_ME_NB_1	Inrogen	I0A[3] = 100,101,102	100 = -1 101 = 23 102 = 2300	
….							
<-			S (12)				подтверждение что получено 12 кадров (происходит либо раз в **k** **(п.4 настоящего документа, либо t2)**

Разбор формата кадра - I (5,1)

I – это формат кадра (всего бывает 3 формата: **I-формат** — информационные кадры (с данными); **S-формат** — подтверждения (сколько кадров принято); **U-формат** — управление (STARTDT, STOPDT, TESTFR)

первая цифра 5 – это количество отправленных кадров сервером

вторая цифра 1 – это кол-во операция подтверждения кол-во принятых кадров клиентом

Заключение

В ходе эмуляции передачи сообщений между Сервером и Клиентом по протоколу МЭК-104, разобран способ создания виртуальных Сервер и Клиент, запущенных на одном ПК и работающих в одной локальной сети. Реализована передача информации от Сервера (порт 2404) к Клиенту по запросу от Клиента. Произведён разбор последовательности передачи информации и структуры пакета МЭК-104, который содержит в себе проверочные суммы передаваемых кадров, команда, адреса датчиков и текущие показания. Протестировали циклическую передачу данных и передачу данных указанных выбранной командой.